

title: Plano de Evolução: Automação Total do DevStack Hub

aula: 01

Plano de Evolução: Automação Total do DevStack Hub

Este documento detalha o roteiro para transformar o DevStack Hub em uma plataforma autônoma de inteligência e documentação.

■ Visão Geral

O objetivo é eliminar a entrada manual de dados. O sistema deve varrer a web, identificar atualizações em frameworks/bibliotecas, analisar a documentação técnica automaticamente e gerar:

1. Dados comparativos atualizados (JSON).
2. Arquivos de Skills (`skill.md`) prontos para consumo por Agentes IA.

■ Fase 1: Ingestão de Dados Automatizada (Crawler Engine)

****Objetivo:**** Coletar documentação bruta de todas as fontes oficiais.

Componentes:

1. ****Docs Spider****: Um web crawler (baseado em Puppeteer ou Playwright) configurado para visitar periodicamente as URLs de documentação oficial.

* ***Alvos:** React docs, Vue docs, MDN, NPM Registry.
2. ****Change Detector****: Sistema que compara hashes de páginas para detectar mudanças de versão (ex: React 18 -> 19).
3. ****Raw Data Lake****: Armazenamento temporário do HTML/Markdown cru extraído.

Automação Proposta:

```
```javascript
// Exemplo conceitual do robô

const targets = ['react.dev', 'vuejs.org', 'tailwindcss.com'];

targets.forEach(async (target) => {

 const docs = await crawl(target);

 if (hasChanges(docs)) {

 triggerPipeline(docs);

 }

});

```
```

■ Fase 2: Processamento Cognitivo (LLM Pipeline)

****Objetivo:**** Transformar texto bruto em dados estruturados e skills.

Pipeline:

1. ****Extração Estruturada**:** Usar um LLM (Gemini 1.5 Pro) para ler a documentação bruta e extrair o JSON padronizado.

* ***Input:** HTML da doc do React.

* ***Output:** Objeto JSON com `pros`, `cons`, `performance\_metrics`, `code\_examples`.

2. ****Skill Synthesis**:** O LLM gera o arquivo `skill.md` otimizado para contexto (System Prompts).

* ***Regra:** "Crie uma skill que ensine um agente a usar a API de Hooks do React baseada nestes novos docs."

■ Fase 3: Expansão de Ecossistema (Stacks & Libs)

****Objetivo:**** Cobrir todo o universo Frontend/Fullstack automaticamente.

Novas Categorias de Monitoramento:

* ****State Management:**** Redux, Zustand, Jotai, Recoil, XState.

- * **Data Fetching:** TanStack Query, SWR, RTK Query.
- * **Routing:** React Router, TanStack Router, Vue Router.
- * **Testing:** Vitest, Jest, Playwright, Cypress, Testing Library.
- * **Build Tools:** Vite, Turbopack, Webpack, Rollup, ESBuild.
- * **Meta Frameworks:** Next.js, Remix, Nuxt, SvelteKit, Astro.

Stacks Automáticas:

O sistema analisará tendências do GitHub e NPM para sugerir "Stacks" automaticamente:

- * **Ex:** Detectou aumento no uso de `Vite + React + Tailwind + Zustand`.
- * **Ação:** Cria automaticamente a entrada "Modern React Stack" no comparador.

■ Fase 4: Integração Contínua (CI/CD for Knowledge)

Objetivo: O site se atualiza sozinho.

1. **GitHub Actions:** Cron job diário.
 - * `00:00`: Roda o Crawler.
 - * `01:00`: Processa novos dados com LLM.
 - * `02:00`: Commita atualização no arquivo `src/data/frameworks.js` e `public/skills/\*.md`.
 - * `02:15`: Deploy automático na Vercel/Netlify.

■ Implementação Imediata (Next Steps)

Para começar esta evolução agora, vamos expandir a base de dados atual manualmente para incluir as bibliotecas solicitadas e adicionar a aba de "Stacks" no sistema.

Ações para o Usuário agora:

1. **Expandir `data/frameworks.js`:** Adicionar 50+ lib/frameworks.
2. **Criar `Stacks` data structures:** Definir combinações populares.
3. **Atualizar Generator:** Simular o botão "Auto-Discover Update".